CLASS ACTIVITY | SOFTWARE ENGINEERING

Functional and Non-Functional Requirements and Modeling

Practice with the QuickBite food delivery case study

5 tasks



What you will be able to do

- **Distinguish** functional requirements, non-functional requirements and constraints
- **Rewrite** vague quality statements as measurable, testable ones
- **Elicit and write** clear requirements from a scenario
- **Build** requirements models: use case, activity, class and sequence diagrams
- **Spot** conflicts between requirements and prioritize them

The case: QuickBite

Scenario

QuickBite connects customers, restaurants and delivery riders. Customers browse restaurants, order food, pay online by card or cash on delivery, and track delivery. Restaurant staff accept or reject orders and update menus. Riders accept delivery jobs and update status. Admins manage restaurants and handle complaints.



Customer



Restaurant staff



Rider



Admin

Key constraints

- 5,000 simultaneous users at dinner peak
- Launch within 6 months
- Card payments follow local financial regulations

Task 1: Classify each statement

Label each one as FR, NFR (name the quality) or Constraint.

1. Customers can search restaurants by cuisine.
2. Search results load within 2 s for 95% of requests.
3. An SMS confirmation is sent after an order is placed.
4. Passwords are stored using salted hashing.
5. The system supports 5,000 concurrent users.
6. Admins can add or remove restaurants.
7. The system is available 99.9% of each month.
8. A customer can cancel an order within 5 minutes.
9. A first-time user orders within 3 minutes, untrained.
10. The system is built with Java and Spring Boot.
11. The system complies with the national card security standard.
12. The system encrypts payment data. (Discuss!)

Task 2: Make NFRs measurable

Quality + **Measure** + **Target** + **Condition**

Rewrite each vague statement using the formula above.

- *The app should be fast.*
- *The system must be secure.*
- *The app should be user-friendly.*
- *The system must be reliable.*
- *The system should scale.*

Tip A good NFR has a number, a load condition and a way to test it.

Task 3: Write requirements

8+ **functional requirements**

FR-01, FR-02, ...

6+ **non-functional requirements**

NFR-01, NFR-02, ...

Peer-review checklist: is it...

- **Unambiguous** one meaning only
- **Testable** you can prove it passes
- **Atomic** one need per statement
- **Necessary** traces to a real need
- **Feasible** achievable in 6 months

Task 4: Model the system



4A

Use case diagram

Actors, use cases, include and extend, plus a written spec for Place Order.



4B

Activity diagram

The order process with swimlanes and decision points.



4C

Class diagram

Classes, attributes and multiplicities.



4D

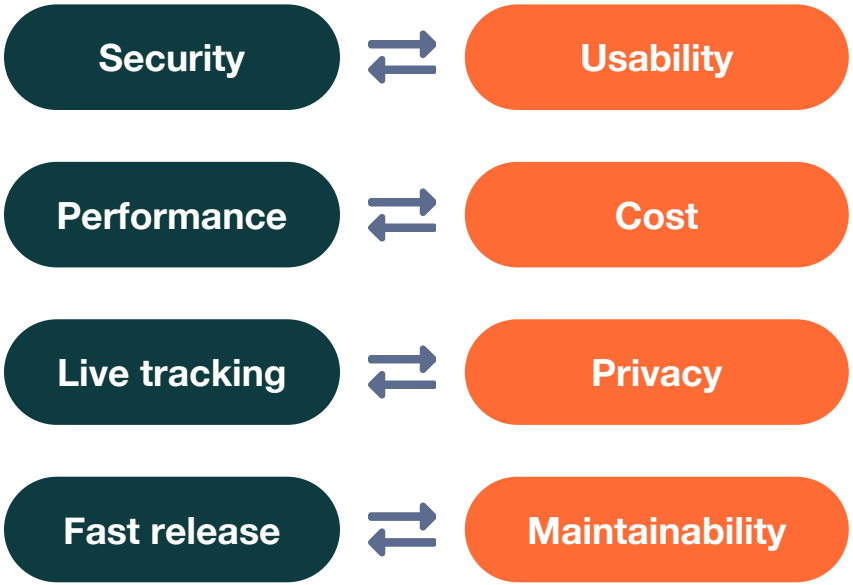
Sequence diagram

Place Order with card payment, including the declined path.

One model per member, then check that all four tell the same story.

Task 5: Conflicts and priorities

Find the conflict. Propose a fix.



Then prioritize with MoSCoW

- **M** **Must** Launch fails without it
- **S** **Should** Important, not vital
- **C** **Could** Nice if time allows
- **W** **Won't** Not in this release